SIMD Advantage Profiling on Apple M5 Max (NEON), single thread

ECSE 4320/6320, Class-wide Project 1. All numbers are from this repository; paths in brackets are the evidence (plots in `plots/`, tables in `data/processed/`, raw CSVs in `data/raw/`, compiler evidence in `vecreports/`). Hypotheses were written before the experiments (`docs/HYPOTHESES.md`); each is marked supported, partly supported or refuted there. Anomalies are labelled unverified where I could not test the cause.

1. Summary

1. SIMD speedup depends on the scalar baseline more than on lane count. Against the assignment-defined `-fno-vectorize` build, f32 SAXPY speeds up 5.2x in L1 and 3.0x in DRAM. Against an explicitly unrolled scalar loop (the primary, conservative baseline) it is 2.1x and 1.35x [`data/processed/exp1_key_table_phase3.csv`]. The default scalar loop runs at one iteration per cycle (loop overhead), so it is a weak baseline [`vecreports/asm/scalar_saxpy_f32.s`].
2. A single thread gets about 150 GiB/s from SAXPY at every size beyond L1, including 2 GiB. I saw no L2, SLC or DRAM boundary; the only step is the L1 boundary at 128 KiB [`plots/exp2_saxpy_locality.png`]. SIMD gain compresses there (2.1x to 1.35x), not at DRAM.
3. fp64 gains almost nothing over unrolled scalar for SAXPY and multiply (1.07x in L1, 1.0x beyond), because two lanes move the same bytes per cycle as the unrolled scalar loop [`plots/exp5_dtype.png`].
4. The f32 dot product is vectorized by plain `-O3`, but with a strict in-order reduction, so it stays bound by the fadd chain: 1.46x over unrolled scalar, flat from L1 to DRAM. With `-ffast-math` it is 13.6x in L1 and 10x in DRAM [`plots/exp1_sweep_f32.png`, `vecreports/VECTORIZATION.md`].
5. 16-byte alignment matters: a vector SAXPY starting at an address that is not 16 B aligned is 40% slower in L1 and 21 to 24% slower beyond [`plots/exp3_alignment.png`].
6. All four kernels are memory bound in DRAM on the roofline (ridge at 1.1 FLOP/B); peak is 146 GFLOP/s fp32 (4 vector FMA pipes inferred) [`plots/exp6_roofline.png`, `plots/exp6_fma_peak.png`].

2. Setup and method

Apple M5 Max, macOS 26.5.2, Apple clang 21.0.0, 6 "Super" and 12 "Performance" cores, 128 B cache lines, 16 KiB pages, L1D 128 KiB and L2 16 MiB (Super tier values from `sysctl`; the SLC size is not exposed) [`docs/ENVIRONMENT.md`]. NEON is 128-bit: 4 x fp32, 2 x fp64, 4 x u32. SME is present but not used (needs streaming mode, not a drop-in SIMD path).

Kernels: SAXPY, elementwise multiply, dot product, 3-point stencil (f32, f64, u32), plus strided SAXPY, gather and triad for Experiments 4 and 6. Each kernel x dtype x build is its own translation unit, called through a function pointer; results are sunk behind compiler barriers [`docs/METHODOLOGY.md`]. Builds: `scalar` (`-O3 -fno-vectorize -fno-slp-vectorize`), `scalar_unroll8` (same plus an unroll-8 pragma, the primary baseline), `auto` (`-O3`), `native` (`-mcpu=native`, which resolves to `apple-m4`), `fastmath` (`-O3 -ffast-math`), and `*_norestrict`, `auto_forcevec` for the negative cases. `-ffp-contract=on` everywhere except fast-math.

Timing: 24 MHz counter (41.7 ns tick), at least 1 ms per sample, 15 samples per point (3 passes x 5 reps, configuration order reshuffled each pass), median with IQR error bars, thermal pressure sampled before and after each sample (no sample was throttled in any run). Call overhead of 0.7 to 0.9 ns is subtracted. The harness thread is set to QoS user-interactive; macOS cannot pin threads.

Effective clock: a dependent integer add chain gives 4.58 GHz (IQR half-width 0.015 GHz), cross-checked by a 3.00-cycle integer multiply chain [`data/raw/freq_phase3.csv`, `docs/METHODOLOGY.md`]. Cycles per element use this value and assume it stays constant (it does not exactly: see below). The distribution is single-mode at user-interactive QoS; `background` QoS runs on a slow cluster (1.3 GHz) [`data/raw/freq_background_probe.csv`]. A powermetrics sample run afterwards [`data/raw/powermetrics_sample_20261006.txt`] shows the Performance clusters top out at 4380 MHz and the Super cluster at 4608 MHz, so a 4.58 GHz chain must have run on a Super core (given the 1-cycle-add assumption); that first sample was idle. Four samples taken during a running SAXPY benchmark [`data/raw/powermetrics_during_benchmark_20261006.txt`, `data/raw/powermetrics_during_benchmark_combined_20261006.txt`] show the Super cluster 100% active at 4608 MHz per busy core (CPU power 14.6 to 14.8 W vs 0.2 to 0.6 W idle) and the Performance clusters at background level; the cluster time-average was 4548 MHz in one sample and 4588 to 4596 MHz in three others. My 4.587 GHz estimate lies in that range, so cycles per element carry about 1% uncertainty. The L1 cliff at 128 KiB (not 64 KiB, the Performance-core L1D) agrees.

Correctness: every kernel x dtype x build over 29 sizes, 7 start offsets and guard regions, against a double-double reference (on this ABI `long double` is only 64 bits, so it adds no precision): at most 1 ulp for SAXPY, mul, triad, strided and gather (observed 0), stencil tolerance 2 ulp (observed 1), exact for u32; dot relative error is bounded by N x eps. A mutation check shows the suite fails for an off-by-one loop bound and a wrong stencil weight [`tests/test_kernels.c`, `scripts/run_tests.sh`]. 64326 checks pass at the final commit.

3. Vectorization verification

Evidence: remarks and loop bodies for every combination [`vecreports/summary.md`, `vecreports/remarks/`, `vecreports/asm/`], annotated in `vecreports/VECTORIZATION.md`.

- SAXPY, mul, stencil, dot (u32 and fast-math): vectorized 4 x 4 (f32, u32) or 2 x 4 (f64): 16 (8) elements per iteration with 4 independent `fmla.4s` (`fmla.2d`), `ldp q` and `stp q` pairs, followed by a 4-element vector loop and a scalar loop; there is no masking on NEON.
- FMA appears as `fmla` (vector) and `fmadd` (scalar) because of `-ffp-contract=on`.
- Scalar builds: "loop not vectorized"; the default `scalar` build is not unrolled (6 instructions per element), `scalar_unroll8` has 8 `fmadd` per 8 elements.
- f32 dot with plain `-O3`: remark "vectorized, width 4, interleave 4", but the body is `fmul.4s` then 16 serial scalar `fadd` per iteration (strict summation order). The assignment statement that a float dot product will not vectorize without fast-math is only partly true on this compiler. The u32 dot uses `mla.4s` with four accumulators without any flag (integer addition is associative).
- Strided SAXPY (stride 2 to 32) and gather: not vectorized by `auto` ("cost-model indicates that vectorization is not beneficial", 4x interleaved scalar). Forced vectorization yields `ld2.4s` de-interleave at stride 2, 2-lane `zip1` at stride 8 and lane inserts `ld1.s {v}[n]` for the gather, always with lane stores [`vecreports/asm/auto_forcevec_*`].
- A process finding: `restrict` on local pointers was ignored (runtime overlap checks and a scalar fallback stayed). It must be on function parameters; the main variants were fixed and

`auto_norestrict` kept as a negative case [`vecreports/VECTORIZATION.md`]. Phase 1 pilot data predate this fix and are not used in any plot.

4. Locality, SIMD gain and data types (Exp 1, 2, 5)

Speedup of `auto` over `scalar_unroll8` (over `scalar`), median ratio [`data/processed/exp1_key_table_phase3.csv`, `plots/exp1_sweep_f32.png`, `plots/exp1_sweep_f64.png`]:

kernel	f32 L1 (32 KiB)	f32 L2 (4 MiB)	f32 DRAM (1 GiB)	f64 L1	f64 DRAM
SAXPY	2.08 (5.18)	1.37 (3.05)	1.35 (3.03)	1.07 (2.63)	1.02 (1.55)
multiply	2.20 (5.17)	1.07 (2.39)	1.17 (2.51)	1.07 (2.52)	1.04 (1.29)
stencil	2.92 (3.77)	2.57 (3.37)	2.54 (3.33)	1.47 (1.95)	1.38 (1.83)
dot, strict `-O3`	1.46 (1.63)	1.52 (1.59)	1.48 (1.59)	1.39 (1.56)	1.40 (1.51)
dot, `-ffast-math`	13.6	11.5	10.1	6.7	5.0

- Locality (Exp 2): f32 SAXPY throughput is 260 GiB/s (0.197 cycles per element) in L1 and 147 to 154 GiB/s (0.33 to 0.35 cycles) beyond, a single step at the L1 boundary and a slow decline to 143 GiB/s at 2 GiB [`plots/exp2_saxpy_locality.png`]. Multiply (three arrays) declines gradually between 128 KiB and 2 MiB. The prediction that gain compresses toward 1x only in DRAM and at the L2 boundary (H2, H3) was refuted for SAXPY and mul f32 and partly supported for f64.
- Data types (Exp 5): the f64 speedup is about half the f32 value in L1 for SAXPY (1.07 vs 2.08) and mul, consistent with lane count; in cycles per element, f64 vector and unrolled scalar are within 7% (0.40 vs 0.43), which points to a memory-operation limit rather than arithmetic (inference from cycle counts, no counters) [`plots/exp5_dtype.png`, `plots/exp5_elements_per_ns.png`]. u32 behaves like f32 (SAXPY 2.65x, mul 2.30x in L1) and its dot product vectorizes without fast-math (5.9x in L1).
- `native` equals `auto` within 3% for all kernels and sizes [`data/raw/sweep_phase3_B.csv`].
- Noise: median CV 1.8% in cache and 1.6% in DRAM, 90th percentile 5%, a few points up to 47% [`data/raw/sweep_phase3_*.csv`].

5. Alignment and tail (Exp 3)

- Alignment [`plots/exp3_alignment.png`]: for the vector build, offsets that are multiples of 16 B run at the offset-0 speed (within 1%). All other tested offsets (4, 8, 12, 20, 28, 60, 124, 132 B) are 40% slower at 32 KiB, 24% at 4 MiB and 21% at 256 MiB, including 8 B offsets and offsets that do not cross a cache line. The unrolled scalar loop varies by at most 6%. Cause unverified (a split of 16 B accesses is a guess). Both arrays were shifted equally; relative misalignment was not tested.

- Tail [plots/exp3_tail_small_n.png, plots/exp3_tail_mod16.png, data/processed/phase4_summary_phase4.txt]: f32 SAXPY costs 0.4 to 1.2 cycles per element for N up to 16, 0.35 at N = 32, 0.25 at N = 64 and 0.19 beyond N = 120; fixed cost is about 2 ns (intercept of an affine fit over N = 32 to 160). Around N = 256, 1024 and 4096 there is no visible dependence on N mod 4 or mod 16 above the roughly 1 ns noise. The disassembly predicts three code paths (N < 4, N < 16, N >= 16); I did not see clear steps at those thresholds. Below N = 64 the curve is irregular and I do not claim an explanation.

6. Stride and gather (Exp 4)

[plots/exp4_stride_gather.png, data/processed/phase4_summary_phase4.txt] (f32, useful bytes counted).

- `auto` equals the unrolled scalar loop at every stride >= 2 (within 10%): SIMD gives nothing. Forcing vectorization is not faster, and 74% slower at stride 4 in L1 (lane stores); at 512 MiB it is 11% faster at stride 2.
- Useful bandwidth in DRAM (512 MiB footprint): 149 (stride 1), 56, 37.5, 20.3, 9.5, 1.0 GiB/s (stride 2, 4, 8, 16, 32); gather 5.1 GiB/s, 39x slower per element than unit stride. In L1 the loss is milder (265 GiB/s at stride 1 to 67 at stride 2, then flat to stride 8).
- Anomaly (unverified): from stride 16 (64 B) to stride 32 (128 B, one element per line) time per element grows 10x (1.17 to 11.5 ns) although the line traffic only doubles; a prefetcher or TLB effect is a guess.

7. Roofline (Exp 6)

[plots/exp6_roofline.png, plots/exp6_fma_peak.png, plots/exp6_bandwidth_scaling.png, data/processed/roofline_points_phase5.csv, data/processed/phase5_summary_phase5.txt]

- Peak compute: 3.99 `fmla.4s` per cycle at 20 independent accumulators = 145.8 GFLOP/s fp32 (IQR 145.7 to 146.9); fp64 72.3 GFLOP/s. Sustained plateau for most accumulator counts is 3.4 per cycle (85%). Four FP pipes inferred (no counters). Uncertainty is dominated by the 4.58 GHz clock estimate (about 0.5%, plus its 1-cycle-add assumption).
- Memory ceiling, one thread: triad 122 GiB/s; SAXPY reaches 150 GiB/s, so triad is not an upper bound for all patterns (both lines are drawn). With 12 concurrent processes the aggregate is 441 GiB/s (3.6x), so one core is the limiting resource, not DRAM. Per-process rates become bimodal (68 to 76 vs 27 to 32 GiB/s), consistent with two core tiers (unverified).
- Intensity (compulsory bytes): SAXPY 0.167 FLOP/B (f32), dot 0.25, multiply 0.083, stencil 0.625; ridge 1.1. All lie left of the ridge: memory bound in DRAM and L2. Fractions of the 146 GFLOP/s peak in DRAM: SAXPY 18%, mul 7%, fast-math dot 20%, stencil 49%; in L1: 32%, 16%, 28%, 57%. In L1 SAXPY and mul sit on the L1 bandwidth slope, not near the FLOP roof. The strict dot is at 3% of peak and 13% of the memory roof everywhere: it is latency bound, neither compute nor memory bound. No kernel is compute bound.

8. Negative cases (Exp 7)

A. Reduction without fast-math [plots/exp7_reduction_accuracy.png, data/raw/accuracy_phase5.csv, vecreports/VECTORIZATION.md]: vectorized multiply, serial

sum, 1.46x over scalar, independent of working set. Fast-math (16 partial sums) is 10 to 14x faster. Numerically the in-order f32 sum has error 3.3, 86, 139, 1.3e5 and 1.1e7 eps at N = 1e3, 1e5, 1e6, 1e7, 1e8 against a double-double reference, because once the sum exceeds 2^24 additions of terms near 1 are absorbed. Fast-math gives 0.5, 1.8, 2.0, 9.4 and 3.8e3 eps. The faster build is also the more accurate one here; its result differs from the strict one by up to 1e-2 relative at 1e7. `scalar` and `auto` strict results differ slightly (139 vs 137 eps at 1e6) because scalar uses fused `fmadd` and auto uses `fmul` plus `fadd`; `native` equals `auto`.

A2. Aliasing [`plots/exp7_alias_restrict.png`, `data/raw/alias_phase5*.csv`, `data/raw/restrict_smalln_phase5.csv`]: without `restrict` the compiler adds a runtime overlap check. For non-overlapping arrays it costs under 7% at large N and 2% at N = 8 to 1024, but 43% at N = 4. If y = x + 1 element the check fails, the scalar path runs, and the loop-carried dependence costs 9.4 to 12.8 cycles per element (28x to 67x slower than 0.19 to 0.34 cycles). Results stay correct (overlap tests).

B. DRAM-bound SIMD: SAXPY f32 at 1 GiB is vectorized perfectly (12 instructions per 16 elements) and is only 1.35x faster than unrolled scalar [`plots/exp1_sweep_f32.png`].

9. AI-resilience checkpoint (Exp 8)

A deliberately naive benchmark was written first, with its corrected twin, and both datasets kept [`src/naive/naive_bench.c`, `plots/exp8_naive_vs_fixed.png`, `data/raw/naive_phase5.csv`, `data/raw/naive_phase5_run1_fix_still_hoisted.csv`, `vecreports/naive/n1_naive_timed_region.txt`].

- N1 (compiler removes the work): a multiply loop in the timing function with no consumer measured 0 ns (the timed region contains no loop in the assembly), reported as infinite GiB/s. Fixed: 263 GiB/s, in line with the harness.
- N3 (first touch in the timed region): 81 vs 148 GiB/s, 1.8x too low.
- N2 (zero-page reads of a calloc array): the predicted artifact did not appear (83 vs 83 GiB/s); the naive variant had a warm-up pass, so this does not prove the artifact cannot occur.
- The first fixed N2 still measured 0 ns: a plain memory-clobber barrier does not stop the compiler from hoisting calls on memory that never escaped (the pointer must be passed to the asm). I kept that dataset as `naive_phase5_run1_*.csv`.
- Unplanned flaws found in my own harness: `restrict` on locals; a non-unrolled `scalar` baseline; `long double` is 64-bit; the last `-Rpass=` option overrides earlier ones (my first remark files were empty). These changed the methodology (see `docs/AI_USE.md`).

10. Anomalies, limitations

- No pinning: a single QoS-based placement; the clock distribution is single-mode (4.58 GHz), and four powermetrics samples taken during a SAXPY run confirm the Super cluster ran it (with the 128 KiB L1 cliff in agreement). The thread migrated among the six Super cores (about 10 to 35% residency per core), so it was not pinned to one core; the effect of migration was not isolated. Only one benchmark was sampled; placement and clock during the other runs are assumed to match (same QoS), not measured.
- No hardware counters. `perf` does not exist on macOS. `xctrace` runs only with `DEVELOPER_DIR` pointing to Xcode, and the exported CPU Counters table had only aggregate `cycle` and bottleneck metrics whose cycle totals disagreed with timing by about 4.7x; I did not use them

(docs/METHODOLOGY.md). Vector-instruction, FMA and cache-miss claims rest on disassembly and timing, not counters.

- Clock estimate assumes a 1-cycle dependent add; cycles per element assume constant frequency.
- Unified memory, 16 KiB pages and 128 B lines are Apple specific; the alignment penalty, flat bandwidth and the stride-32 jump may not generalize to x86 (no x86 host was provided, Phase 5b not run).
- Thermal state is a coarse pressure level, not a temperature; sessions were short (about 10 minutes each) so sustained-throttle behavior is untested. Background load was not controlled beyond what I observed (load average 1.2 to 2.4).
- Unexplained: single accumulator `fmla.4s` latency (6.2 cycles vs about 4 with more accumulators); non-monotonic FMA throughput vs accumulator count; the dip near 16 to 32 KiB in the f32 SAXPY curve; the stride-32 cliff; the alignment mechanism; no SLC or L2 boundary for one thread.

11. Reproducibility

`scripts/run_all.sh` rebuilds, runs the tests, regenerates vectorization reports and all 15 plots from the committed raw data. `COLLECT=quick scripts/run_all.sh` re-measures a reduced version of every experiment in a fresh clone (result recorded in `docs/REPRODUCTION.md`); `COLLECT=full` repeats the full measurement (about 40 minutes). Raw data: one CSV per run in `data/raw/` with timestamp, session, build, kernel, dtype, N, offset, stride, rep, time and thermal state.